Payment Processing Platform

Documentation for the merchant payments technical exercise

Manjunath Inti

22 October 2026

github.com/intimanjunath/Merchant-payment-processing-platform (branch `main`)

Sections 2 to 7 are the six documentation deliverables listed in section 6 of the brief, in the same order and under the same headings.

Section 8 goes through the rest of the brief in its own order: what is evaluated (§2), the functional requirements (§3), the observability and operational requirements (§4) and the technical expectations (§5). Each item is answered directly, with a pointer to where it is implemented.

The table on the next page maps every part of the brief to the section that answers it.

The repository's `README.md` covers the same material in shorter form. `design_doc.md` has the full design, including the failure-mode analysis, and `project_spec.md` has the exact contracts (API, schema, metrics, configuration).

Where each part of the brief is answered

Brief	Answered in	Summary
§6 Setup instructions	Section 2	Docker only; one command; sign-in details and test cards
§6 Architectural overview	Section 3	API, worker and dashboard over one PostgreSQL database
§6 Tradeoffs made	Section 4	Seven decisions, with what each one cost
§6 Assumptions, §7 "write them down"	Section 5	Currency, amounts, refunds, settlement, users, provider guarantees
§6 Production considerations	Section 6	What is local-only today and what replaces it
§6 Areas for future improvement	Section 7	Six next steps, in priority order
§2 What we're evaluating	8.1	System design, code quality, decisions, operations, tradeoffs
§3.1 Backend: APIs, authentication, lifecycle, async workflow	8.2	Endpoints, how a request is validated and how a login expires
§3.2 Frontend	8.2, Appendix B	React dashboard; screenshots
§4.1 Logging	8.3	JSON logs, request ID into the worker, redaction
§4.2 Metrics	8.3	RED, payments by state, queue depth, lag, retries
§4.3 Tracing	8.3	W3C trace context carried from the request into the job
§4.4 Health and lifecycle	8.3	`/health`, `/ready`, audit row per transition
§4.5 Error handling and resilience	8.3	One error shape, idempotency keys, retries, dead letter
§5 Technical expectations	8.4, Appendix A	All nine areas; testing strategy

1 What I built

A small payment platform for merchants. A merchant signs in, creates a card payment, captures it, waits for it to settle, and can refund it. There is a REST API, a background worker that handles settlement and anything the provider did not answer clearly, and a web dashboard.

The brief asks for a small, well-built slice rather than a broad one, so I kept the feature set narrow and spent the time on correctness when things fail. A retried request must not charge twice. A provider timeout must not leave a payment stuck. A process that stops halfway through must be picked up and finished exactly once. Every action is idempotent, every change of state is audited, and anything that cannot be resolved automatically is paused for review instead of being retried forever.

Everything runs locally with one command. The payment provider is simulated, so approvals, declines, timeouts and lost responses can be reproduced without an external account.

In scope	Left out
Merchant sign-in for the dashboard and for API clients	Customer checkout and card tokenization (the provider's job)
Create, capture, full refund, details, list and search	Partial captures and refunds, cancellation, multiple currencies
Settlement confirmation and recovery of uncertain outcomes	Outbound webhooks, reconciliation reports
Idempotency, audit trail, logs, metrics, trace propagation, health checks	A deployed tracing backend and alerting
Local deployment with Docker Compose	TLS and production infrastructure (section 6)

2 Setup instructions

You need Docker with Compose v2 (Docker Desktop includes both) and free ports 5173, 3000, 3001, 5432 and 5433. Node.js is not needed on the host: the build, the migrations and the tests run in containers.

2.1 Start the platform

```
git clone https://github.com/intimanjunath/Merchant-payment-processing-platform.git
cd Merchant-payment-processing-platform
cp .env.example .env
docker compose up --build
```

The first build takes a few minutes. `.env.example` holds development-only values; the copy is ignored by git. To check that everything is up, run `docker compose ps` in a second terminal: `api` and `worker` should be `healthy` and `web` `running`.

2.2 Sign in

Open **http://localhost:5173**. There are two development merchants, each with a few sample payments in different states.

Merchant	Email	Password
Acme Coffee	`merchant1@example.test`	`acme-coffee-dev`
Blue Bicycle Shop	`merchant2@example.test`	`blue-bicycle-dev`

2.3 Take a test payment

1. Click **New payment**, enter an amount and a reference, and choose **Visa ending 4242**. The payment opens as **Authorized**.
2. Click **Capture** and confirm. Within about 15 seconds the page shows **Settled** without a reload.
3. Click **Refund** and confirm. The payment is **Refunded**, and its history lists every step.

The other test cards exercise the failure paths. Real card numbers are rejected; only these tokens are accepted.

Test card	Token (API)	What happens
Visa ending 4242	`pm_card_visa`	Approved
Visa ending 0002	`pm_card_declined`	Declined; the payment is Failed
Mastercard ending 4444	`pm_timeout`	The request never reaches the bank; about 10 seconds later the worker marks it Failed
Amex ending 0005	`pm_lost_response`	Approved, but the response is lost; about 10 seconds later the worker confirms it Authorized

2.4 Run the checks and tests

```
docker compose run --rm dev npm ci          # install packages once, after a fresh clone
docker compose run --rm dev npm run check   # format, lint, type-check, build
docker compose run --rm dev npm test        # all backend and dashboard tests
```

2.5 **Endpoints**

Service	URL
Dashboard	http://localhost:5173
API	http://localhost:3000 (also http://localhost:5173/api)
API reference (OpenAPI 3.1)	http://localhost:3000/openapi.json
API health, readiness, metrics	http://localhost:3000/health, /ready, /metrics
Worker health, readiness, metrics	http://localhost:3001/health, /ready, /metrics

`docker compose down` stops the platform and keeps the data; `docker compose down -v` also deletes it.

3 Architectural overview

3.1 Components

There are three processes. The API serves the dashboard and API clients. The worker runs background jobs. The dashboard is a React app served by nginx, which also forwards `/api` to the API so the browser talks to a single origin. All three use one PostgreSQL database, which holds the payments, the job queue and the audit trail. The simulated provider keeps its data in a separate schema and commits in its own transactions, so it behaves like an external system.

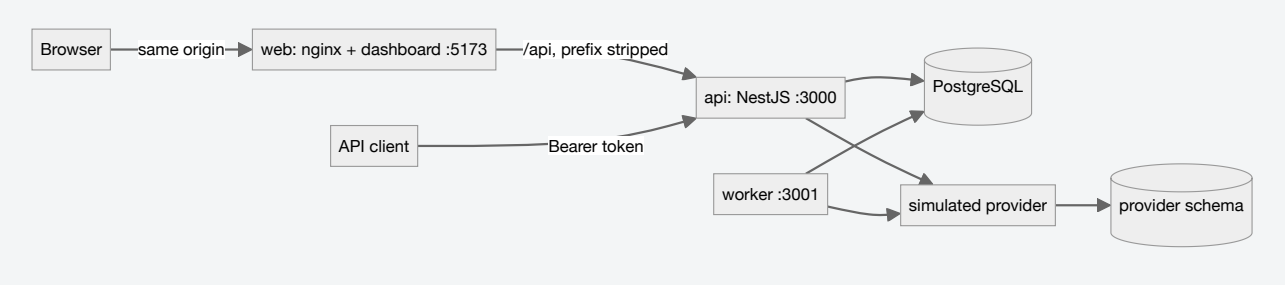


Figure 1. Processes and their connections.

The apps are thin; the rules live in shared packages.

Folder	Built with	Responsibility
`apps/api`	NestJS 11	Sign-in, validation, payment endpoints, OpenAPI, health and metrics
`apps/worker`	Node.js	Background jobs: recovery of uncertain provider outcomes, settlement confirmation
`apps/web`	React 19, MUI 9, Vite, nginx	The merchant dashboard
`packages/payments`	TypeScript	Every lifecycle rule; the API and the worker only call it
`packages/db`	PostgreSQL 18, `pg`	All SQL, migrations and seeds
`packages/provider`	TypeScript	The provider interface and the simulated provider
`packages/observability`	Pino, Prometheus client	Logging with redaction, metrics, request and trace context

3.2 How a payment request is processed

A provider call can be slow, can fail, or can succeed without the response arriving. Holding a database transaction open across it would let one slow response block other payments. Calling first and writing afterwards would lose track of a charge if the process stopped in between. So every create, capture and refund goes through three steps:

1. **Reserve.** One short transaction records the operation and a recovery job for it (and, for a new payment, the payment and its first audit row), keyed by the client's `Idempotency-Key`. For a capture or refund the payment row is locked first, so two concurrent requests cannot both proceed. This commits before the provider is called.
2. **Call the provider** with no lock held.
3. **Finalize.** A second transaction writes the outcome, only if this process still owns the job (its generation number is unchanged) and the payment is still at the version it had at reservation (compare-and-set).

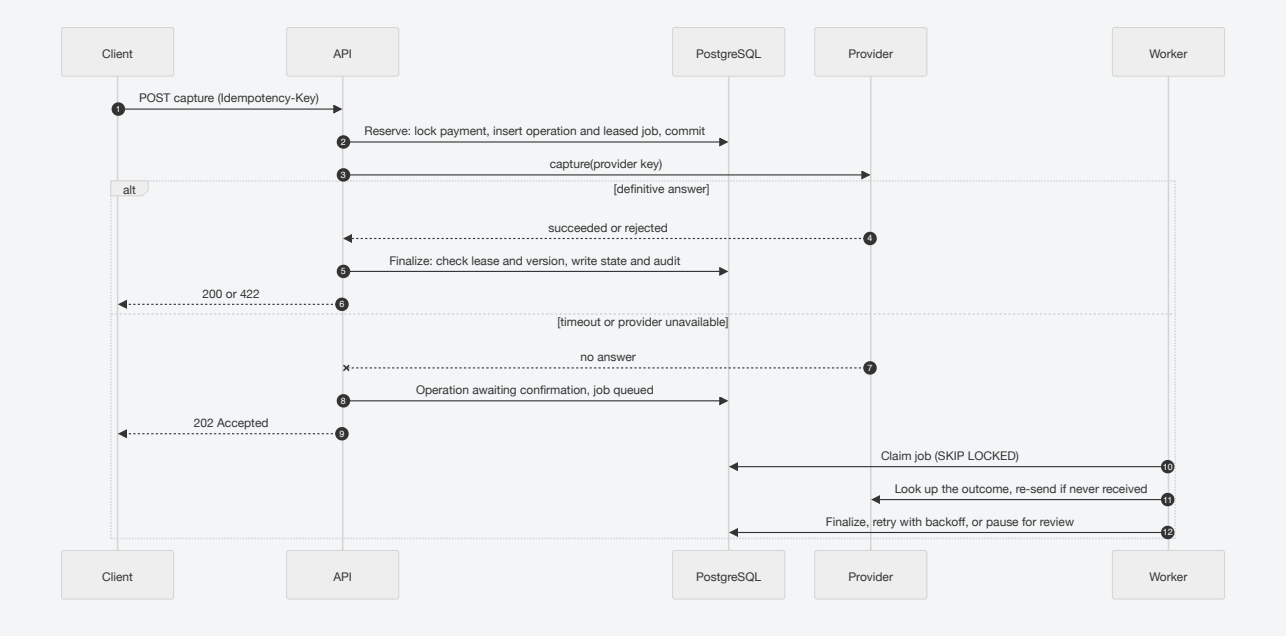


Figure 2. A capture: definitive answer on top, provider timeout below.

If the provider does not answer, the API returns `202 Accepted`, leaves the payment in its current state and queues the job; the worker then asks the provider for the real outcome. If the process stops at any point, the reserved job is still in the database, its lease expires and the worker picks it up. Repeating a request with the same key returns the stored response instead of acting again.

3.3 **Payment lifecycle**

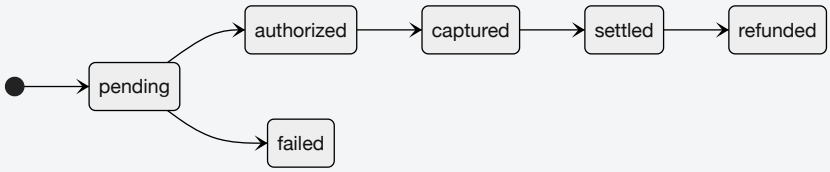


Figure 3. Allowed transitions.

Captures and refunds are for the full amount. Any other move is rejected with `409 INVALID_STATE_TRANSITION`. The transition table is in `packages/payments/src/transitions.ts`.

3.4 **Background work and failure handling**

The worker claims due jobs with `SELECT … FOR UPDATE SKIP LOCKED` and holds each under a lease with a generation number. Two workers never claim the same job at once. If a worker stalls and its lease expires, another worker takes the job over, and the stalled worker's late result is discarded by the generation check.

Job or rule	Behaviour
Recovery	Asks the provider for the outcome of an uncertain action; re-sends a capture or refund the provider never received
Settlement	Confirms that captured funds are available, about ten seconds after capture
Retries	A timeout or unreachable provider is retried three times with backoff (about 2, 4 and 8 seconds, with jitter)
Dead letter	When retries run out, or the work is unresolved after 30 minutes, the job is paused. A create, capture or refund becomes `needs_review`; a settlement is flagged for review. Paused work shows in the dashboard and in `jobs_paused`, and blocks other actions on that payment
Shutdown	The worker stops claiming and drains in-flight jobs for up to 15 seconds; anything left is reclaimed when its lease expires

3.5 Authentication

Sign-in (`POST /auth/login` for the dashboard, `POST /auth/token` for API clients) checks an Argon2id password hash, creates a session row and issues an HS256-signed JWT with the merchant ID, session ID, issuer, audience and expiry. The dashboard receives it in an HttpOnly cookie and sends a CSRF token with every change; API clients send it as `Authorization: Bearer`.

Each request is checked for signature, algorithm, issuer, audience and expiry, and then the session row must be active and belong to that merchant. A login lasts eight hours and ends at once on sign-out, which revokes the session. Failed sign-ins are throttled per email and per IP address. Every query is scoped to the signed-in merchant; another merchant's payment returns 404.

3.6 Deployment

One multi-stage Dockerfile builds the workspace once. Its `app` image runs the API, the worker and a one-off `init` job; its `web` image is nginx with the built dashboard. Compose starts the database, then `init` (migrations and development data), then the API and worker once `init` succeeds, then the dashboard once the API is ready.

4 Tradeoffs made

What I chose, why, and what it cost.

PostgreSQL as the job queue, not a message broker

A job is written in the same transaction as the payment change that needs it: both commit or neither does. With a separate broker there is a gap between committing the payment and publishing the message, which needs an outbox table to close. It is also one less service to run.

Cost: a broker's throughput and tooling. Claiming, leases, retries and lease expiry are implemented here, and queue throughput is bounded by the database.

Reserve, call, finalize in separate transactions

Makes every mutation idempotent and recoverable after a crash, and keeps locks out of network calls.

Cost: the simplicity of one transaction per request. An operation can be reserved but not yet finalized, and that intermediate state needs its own handling and tests.

A JWT backed by a server-side session

A signed token alone cannot be revoked before it expires. The session row gives a fixed expiry and makes sign-out immediate.

Cost: stateless authentication. Every request does one session lookup.

Hand-written SQL instead of an ORM

Correctness here depends on transaction and lock boundaries. Plain SQL in one package keeps them visible and reviewable.

Cost: generated queries and other ORM conveniences.

A simulated provider

Every success and failure path is reproducible in the dashboard and in tests, including a timeout before the charge and a lost response after it.

Cost: a real provider integration. The provider interface is where one would plug in.

Full captures and refunds, US dollars only

One refund path and one state diagram, which left time for the failure handling.

Cost: partial amounts and multiple currencies.

One Dockerfile and Docker Compose

One build for every Node service and one command to run the platform, with nothing to install but Docker.

Cost: production orchestration (section 6).

5 Assumptions

Decisions made where the brief was open:

- **Money.** US dollars only, stored as integer cents, from $0.01 to $1,000,000.00.
- **Amounts.** Captures and refunds are for the full amount. No partial amounts, cancellations or voids.
- **Refund timing.** Refunds are allowed only after settlement, which keeps a single refund path. Real providers usually also allow refunding a captured payment, sometimes as a reversal before settlement.
- **Settlement.** Provider-confirmed availability of funds, simulated ten seconds after capture. Not a bank payout.
- **Lost authorizations.** An authorization the provider never received fails; it is not retried later. Charging a customer long after the request is a business decision, so I left it out.
- **Users.** One user per merchant account, no staff roles. References need not be unique; idempotency keys prevent duplicates.
- **Card data.** Never reaches the platform. Checkout and tokenization belong to the provider; payments take only provider test tokens.
- **Provider guarantees.** The provider deduplicates requests by key, reports the outcome for a key, and never lets a request it reported as "not received" succeed later. The simulated provider implements exactly this; a real provider must be checked against it.
- **Environment.** Runs locally over HTTP; the sample merchants and payments are development data.

6 Production considerations

The application design does not need to change for production: the API is stateless, workers scale horizontally, and nothing depends on local memory. The environment around it does. In priority order: TLS and a secrets manager; a real provider checked against the assumptions in section 5; a managed database with tested restores; CI and alerting.

6.1 Known limitations of the local setup

Limitation	Effect	Fixed by
Plain HTTP (`COOKIE_SECURE=false`)	Local use only; the API refuses `NODE_ENV=production` without secure cookies	TLS
One shared per-IP login limit behind nginx	All browser users count against one limit; the per-email limit still applies	Trusted proxy configuration
Simulated provider	No real money moves	A real provider adapter
No resume tool for paused work	Resolving `needs_review` needs a manual decision	Operator tooling
Development secrets in `.env.example`	Not for any shared environment	A secrets manager

6.2 What changes for production

Area	Now	Production
Transport	HTTP	TLS at a load balancer; `COOKIE_SECURE=true`; `TRUST_PROXY` set to that load balancer
Secrets	`.env` file	A secrets manager (for example AWS Secrets Manager or HashiCorp Vault) with rotation
Runtime	Docker Compose	Kubernetes or Amazon ECS; at least two API and two worker replicas; health-gated rolling deploys
Database	One PostgreSQL container	Managed PostgreSQL (for example Amazon RDS or Cloud SQL) with high availability, point-in-time recovery and tested restores; PgBouncer
Migrations	Run at container start	A separate, gated release step
Provider	Simulated	A real provider adapter (for example Stripe), checked against section 5
Network	All ports on localhost	Only the dashboard and API behind the load balancer; metrics, health and OpenAPI internal only
Logs	stdout	Shipped to a log platform (for example Loki, Elasticsearch or Datadog)
Metrics and alerts	`/metrics` endpoints	Prometheus and Grafana; alerts on `jobs_paused` > 0, queue lag, provider timeouts, 5xx rate, stale worker heartbeat
Tracing	Trace IDs in logs	OpenTelemetry SDK and collector, with Tempo or Jaeger
Delivery	Checks run by hand	CI (for example GitHub Actions): checks, both test suites, image build, dependency and image scanning
Data lifecycle	Everything kept	Retention for completed jobs, sessions, login attempts and idempotency records

Capacity	Not load-tested	Load tests (for example k6) of listing, job claiming and connection pools
Operations	Paused work found with SQL	Runbooks, a guarded resume tool, on-call ownership

PostgreSQL carries the data, the queue and the audit trail, so it is the first bottleneck. Under measured load the next steps are connection pooling, retention for old jobs and idempotency records, read replicas for listing, and a dedicated broker only if job throughput outgrows polling.

7 Areas for future improvement

In priority order:

1. **CI pipeline** running the checks, both test suites and the image build on every change. They are run by hand today.
2. **Guarded resume tool for paused work,** so a `needs_review` payment can be resolved without hand-written SQL, with the decision recorded in the audit trail.
3. **Browser end-to-end tests** (Playwright) for create, capture, settle and refund, and for reloading during a lost response.
4. **Tests for the remaining crash points:** before the reservation, before the provider call, and after the commit but before the response. The design covers them; no automated test does yet.
5. **Real provider adapter** (for example Stripe) behind the existing interface, and an OpenTelemetry tracing backend.
6. **Wider payment features:** partial captures and refunds, refunds before settlement, more currencies, outbound webhooks, API keys and staff roles.

8 Response to the brief

The rest of the brief, item by item, in its own order and wording.

8.1 What we're evaluating (§2)

Criterion	What I did	Details
System design	API, worker and dashboard over one PostgreSQL database. Each create, capture and refund is reserved, sent to the provider with no lock held, and finalized, so a timeout, a retry or a crash at any step has a defined outcome	3.1–3.4
Code quality	Strict TypeScript; ESLint and Prettier enforced by `npm run check`. Lifecycle rules only in `packages/payments`, SQL only in `packages/db`. 456 Jest and 94 Vitest tests, all passing	8.4, App. A
Architectural decisions	PostgreSQL as the job queue; reserve, call, finalize; the worker resolves uncertain outcomes; one package for business rules; server-side sessions	3, 4
Operational thinking	JSON logs with request and trace IDs carried into the worker; RED, business and queue metrics; health and readiness; an audit row per transition; idempotency, retries with backoff and a dead-letter state	8.3
Engineering tradeoffs	Seven tradeoffs with their costs; production gaps listed rather than hidden	4, 6

8.2 Functional requirements (§3)

Backend APIs

Brief	Implementation
Log in to a merchant account	`POST /auth/login` (dashboard, HttpOnly cookie); `POST /auth/token` (API clients, Bearer token); `POST /auth/logout`
Create a payment	`POST /payments` with `Idempotency-Key`
Retrieve payment details	`GET /payments/{id}`, including state history and every provider request
Capture a payment	`POST /payments/{id}/capture` with `Idempotency-Key`
Refund a payment	`POST /payments/{id}/refund` with `Idempotency-Key`
List / search payments	`GET /payments`: status filter, reference search, cursor pagination

Authentication: the three questions in the brief

Brief	Answer
Approach chosen	A signed token issued at login (HS256 JWT) backed by a session row in PostgreSQL. Cookie plus CSRF token for the browser; Bearer header for API clients
How a request is validated	Signature, algorithm, issuer, audience and expiry are checked; then the session must exist, be unrevoked and belong to the token's merchant. The merchant ID comes only from the session, never from the request
How a login expires	After 8 hours (`SESSION_TTL_SECONDS`), or immediately on logout, which revokes the session row

Lifecycle and asynchronous workflow

Brief	Implementation
Pending → Authorized → Captured → Settled, with Failed and Refunded	Exactly these six states; any other move returns `409 INVALID_STATE_TRANSITION` (3.3)
At least one asynchronous workflow	Settlement processing: the worker confirms captured funds and moves the payment to Settled. A second workflow recovers create, capture and refund calls whose outcome was uncertain (3.4)

Frontend

Brief	Implementation
Logging in to a merchant account	Sign-in page; the session survives a reload and ends on sign-out or expiry
Viewing payments	Payments list, newest first, 10, 25 or 50 per page
Filtering / searching	Status filter and reference search, kept in the URL
Viewing payment details and statuses	Details page with lifecycle progress, full history and provider requests; refreshes itself while work is pending
Beyond the brief	Create, capture and refund from the dashboard, safe against double clicks and lost responses

8.3 Observability and operational requirements (§4)

4.1 Logging

Brief	Implementation
Structured logs	JSON lines (Pino), each with `event`, `service`, `requestId` and, when known, `traceId`, `paymentId`, `jobId`
Correlation / request ID across API and async workflow	`X-Request-Id` accepted or generated, echoed in the response, stored on the operation and its job; the worker logs under it. One search for a request ID shows the API request, the provider call, every worker attempt and the settlement
Log levels used deliberately	`debug` routine polling and claims; `info` state changes; `warn` timeouts, retries, lost leases; `error` work paused for review
No PAN, CVV or secrets in logs	Card numbers and CVVs are never accepted, only provider tokens; only brand and last four are stored. Passwords, tokens, cookies and secrets are redacted by key at any depth. Request and response bodies are never logged

4.2 Metrics

Brief	Implementation
Request rate, error rate, latency	`http_requests_total` and `http_request_duration_seconds` by method, route and status class
Payments created, captured, refunded, failed, by state	`payment_transitions_total{from_state, to_state}` counts each event; `payments_by_state` gives current totals
Queue depth, processing lag, retry and failure counts	`jobs_due`, `jobs_lag_seconds`, `job_runs_total{result}` (retried, paused, failed and others), `jobs_paused`

Labels never include payment IDs, merchant IDs or references, so the number of series stays bounded.

4.3 Tracing

Brief	Implementation
A trace spanning the API request and the async work	W3C `traceparent` accepted or started per request, stored on the operation and job; the worker continues the same trace ID with a new span linked to the request's span. Trace and span IDs are in every log line
How distributed tracing would be adopted	Add the OpenTelemetry SDK and a collector exporting to Tempo or Jaeger. The trace context already stored on jobs becomes the parent of the new spans, so no data model change is needed

4.4 Health and lifecycle

Brief	Implementation
Health / readiness endpoints	API: `/health` (process up), `/ready` (database reachable, migrations applied). Worker: `/health` (heartbeat within 30 seconds), `/ready` (also false once shutdown starts). Checks are time-bounded
Audit trail of state transitions	One row per transition: from and to state, actor (`merchant:<id>` or `system:settlement`), source (API or worker), reason, request ID and time. Written in the same transaction as the change and shown on the details page

4.5 Error handling and resilience

Brief	Implementation
Consistent error responses	One shape: `{"error": {"code", "message", "requestId"}}`, with stable codes such as `VALIDATION_ERROR`, `NOT_FOUND`, `INVALID_STATE_TRANSITION`, `IDEMPOTENCY_CONFLICT`, `PROVIDER_REJECTED`. Messages never echo submitted values
Idempotent create, capture, refund	`Idempotency-Key` required. A repeat returns the stored response; the same key with a different body returns `409 IDEMPOTENCY_CONFLICT`. The provider is called with a key derived from the operation, so it deduplicates too
Retries, timeouts, failure handling	5-second provider timeout; three retries with exponential backoff and jitter; then dead letter: the job is paused, the payment marked `needs_review`, counted in `jobs_paused` and shown in the dashboard

None of this needs a full observability stack to run locally; section 6.2 lists what connects to these seams in production.

8.4 Technical expectations (§5)

Area	Implementation	Where
Architecture and code organization	Three apps over shared packages; lifecycle rules only in `payments`, SQL only in `db`	`apps/`, `packages/`
API design	REST resources, one error shape, idempotency keys on every mutation, cursor pagination, OpenAPI 3.1 checked against real responses in tests	`/openapi.json`

Frontend quality	Loading, empty and error states; works at phone width; list, details, actions and session logic in plain TypeScript controllers with unit tests	`apps/web/src/lib`
Scalability	Stateless API; workers scale out with `SKIP LOCKED`; keyset pagination; trigram index for reference search; no lock held during provider calls	6.2
Security awareness	Argon2id passwords; HttpOnly cookie, CSRF token and origin check; login throttling; tenant isolation (404 across merchants); tokens only, no card data; log redaction; containers run as non-root	3.5
Operational maturity	Health and readiness, metrics, structured logs, idempotency, retries with dead letter, graceful shutdown	8.3
Testing strategy	Integration tests on real PostgreSQL, plus contract, concurrency, fault-injection and crash-recovery tests	Appendix A
Maintainability	Strict TypeScript, enforced lint and format, small modules, decisions and contracts documented in `design_doc.md` and `project_spec.md`	`npm run check`
Observability and logging	See 8.3	8.3

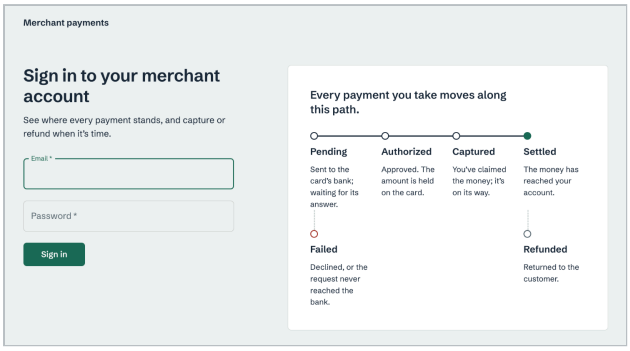
Appendix A. Testing: what is tested, and why

Most backend tests run against a real PostgreSQL instead of mocks, because the guarantees come from constraints, row locks, `SKIP LOCKED` and transaction boundaries; a mock would not exercise them. The tests use a separate database with durability turned off for speed; locking behaves the same.

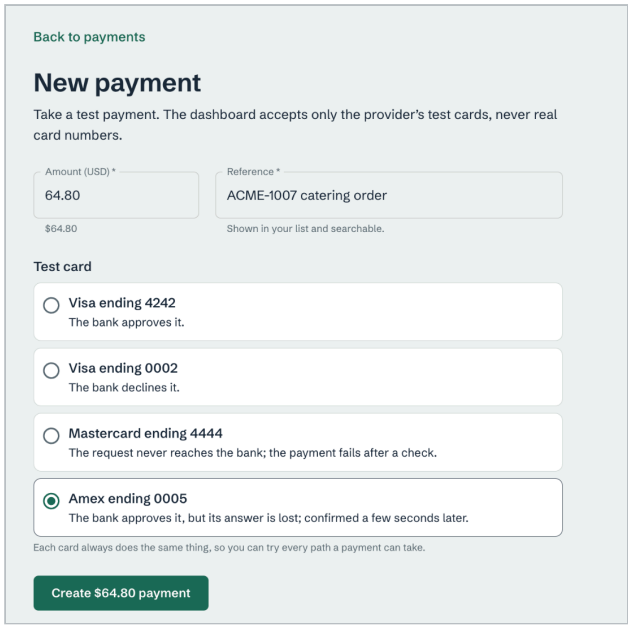
Kind	What it covers	Why
Unit	State machine, request hashing, cursor signing, retry delays, log redaction, configuration	Pure rules that everything else relies on
Integration	API and worker flows end to end, every test card, login expiry and revocation, CSRF, throttling, tenant isolation	Proves the flows a merchant uses, on the real database
Contract	Every API response validated against the OpenAPI document; every route documented	Keeps the published API honest
Concurrency	Two captures at once make one provider call; two workers never claim the same job; same-key creates make one payment	Double charges come from races
Fault injection	Timeout before and after the provider's effect, lost response, provider unavailable, retries exhausted	The failure paths are where payments go wrong
Crash recovery	Provider succeeds and the process stops before saving: recovered exactly once. A stale lease owner cannot overwrite the new one	The hardest case to get right by inspection
Dashboard	Session, list paging, details polling, pending-action rules (Vitest)	Behaviour that survives reloads and lost responses
Smoke	Fresh clone, `docker compose up --build`, healthy services	Setup instructions work as written

> **Latest run, 22 October 2026:** `npm run check` passes. Jest 34 suites, 456 tests passed. Vitest 9 files, 94 tests passed. Not automated yet: browser end-to-end tests, component tests, load tests, coverage reporting.

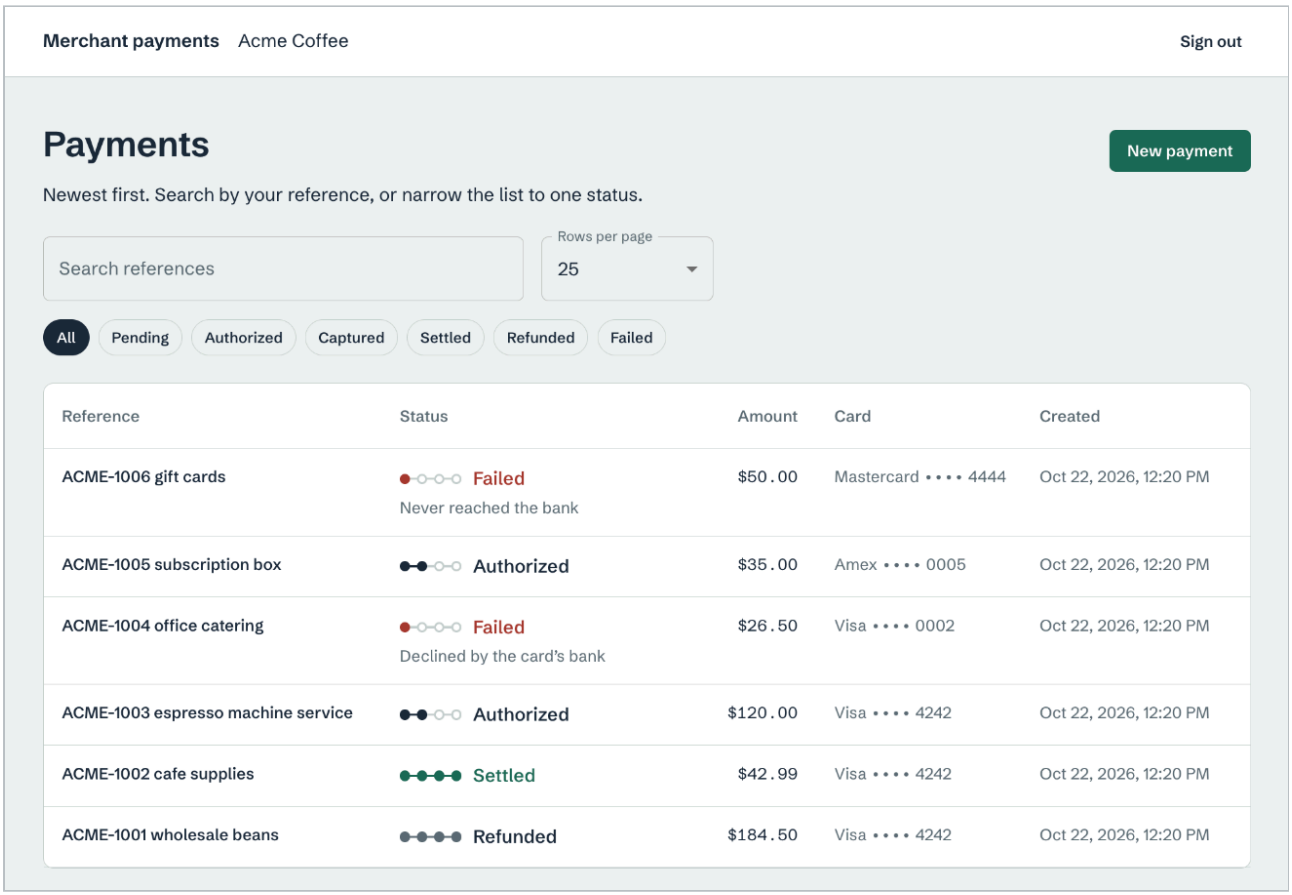
Appendix B. Dashboard screenshots



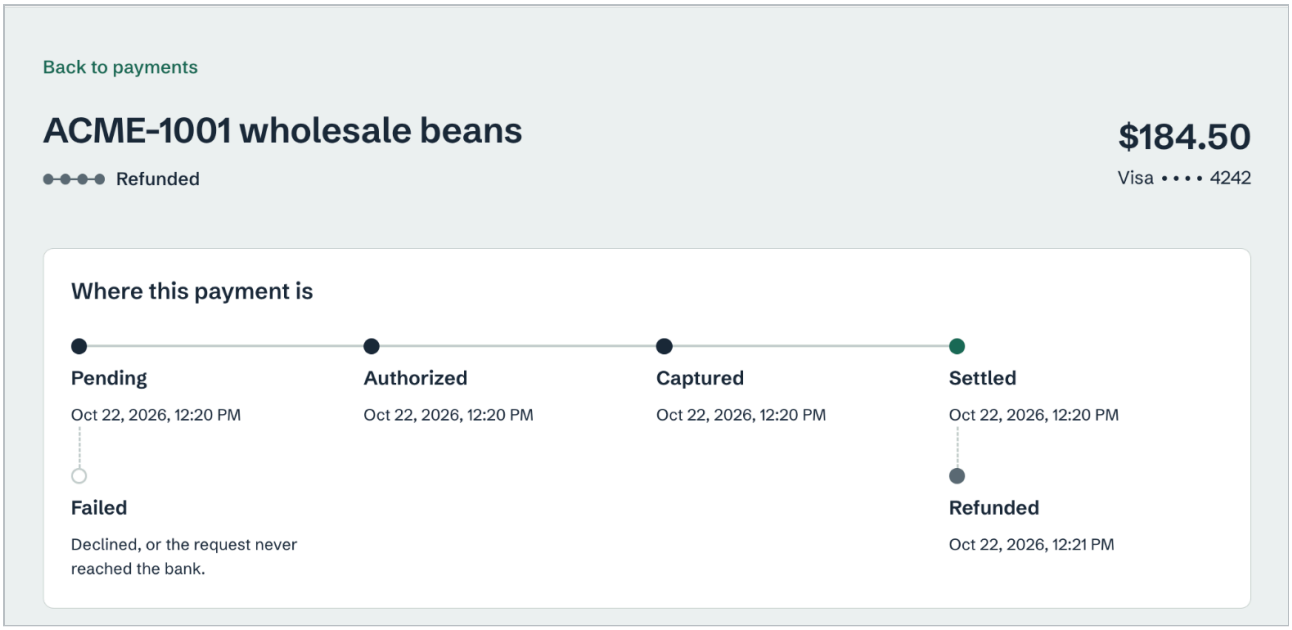
Sign in. The payment lifecycle is explained next to the form.



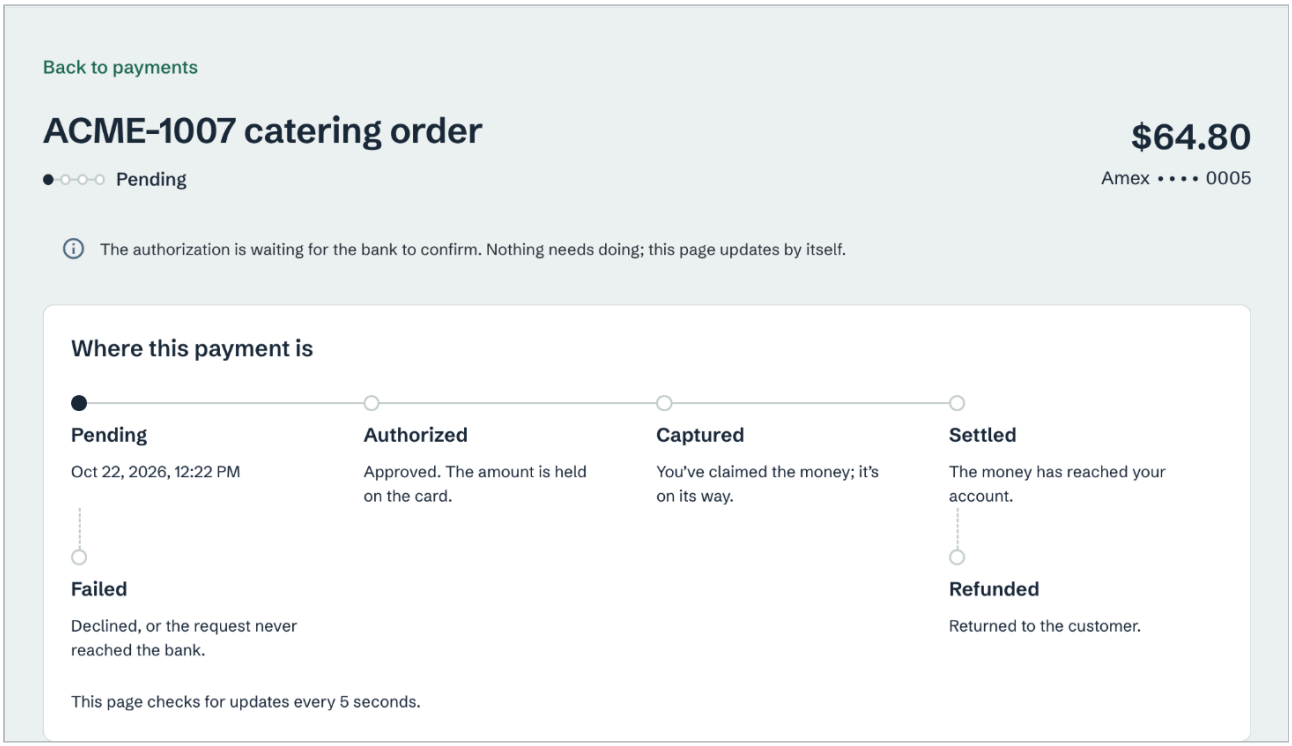
New payment. One test card for each path a payment can take.



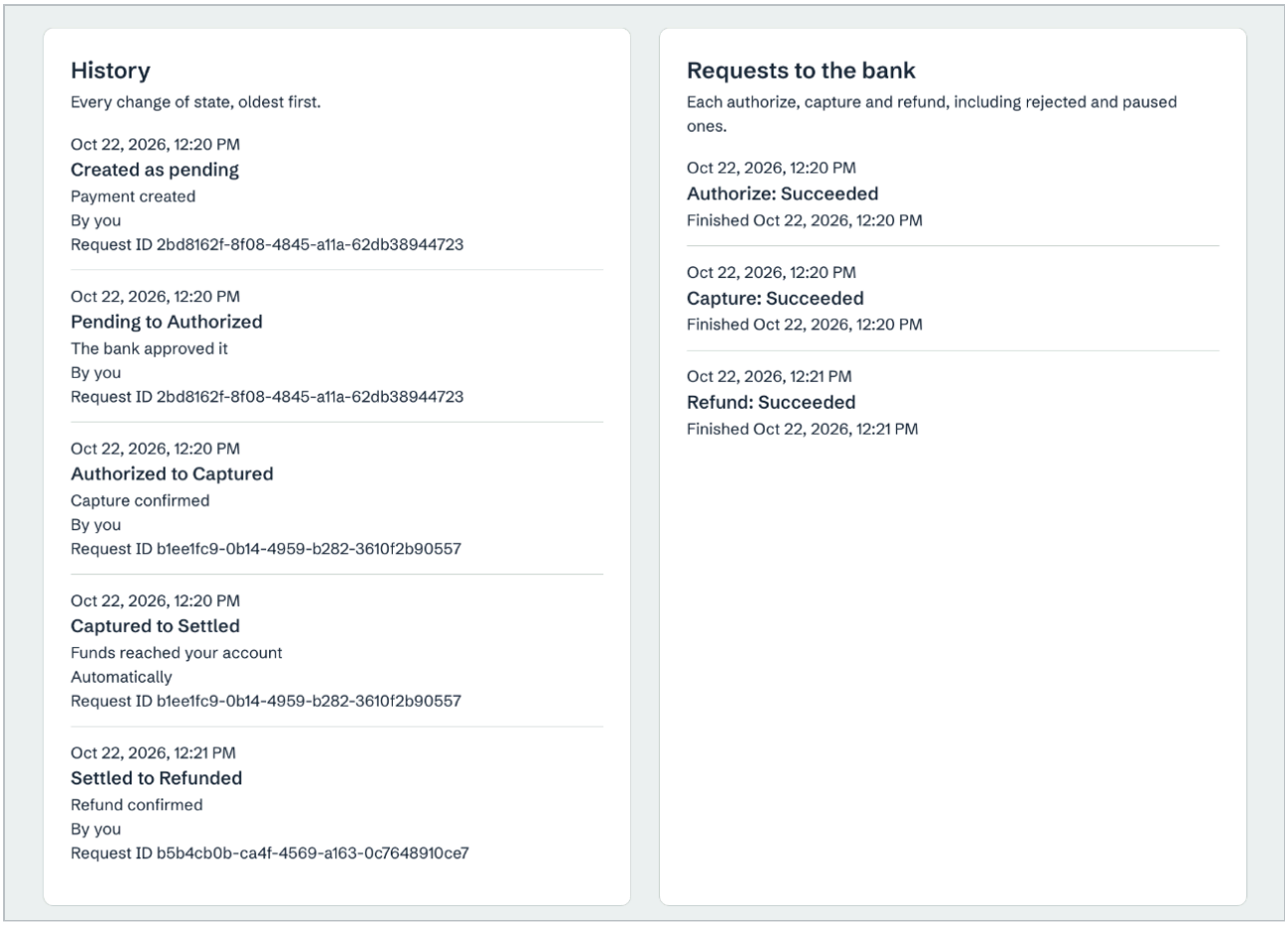
Payments. Status filters, reference search and paging; each row shows how far the payment has progressed.



Payment details. Current state and when each stage was reached.



Recovery in progress. The provider's response was lost; the worker confirms the outcome and the page updates itself.



History. Every state change with actor, reason and request ID, and every request sent to the provider.